

Personal Library Catalogue Generator – Requirements (Version 1)

Overview

Develop a Python application that generates a fully static website for my personal collection of physical books.

The application should run entirely on my local machine and produce a self-contained static website that can be opened directly from `index.html` without requiring a web server.

The generated website must also be suitable for deployment to GitHub Pages in a future phase without requiring architectural changes.

This is **not** intended to be a full library management system. There will be no database, user accounts or backend services. The website is generated entirely from a spreadsheet.

Objectives

The application shall:

- Read book information from an Excel spreadsheet.
- Validate the data before generation.
- Generate a JSON representation of the catalogue.
- Generate a responsive static website.
- Generate an individual HTML page for every book.
- Copy all required assets into the output directory.
- Rebuild the entire site from a single command.

Example:

```
python generate.py
```

Running this command should completely regenerate the website.

Technology Stack

Language

- Python 3.12+

Libraries

- pandas
- openpyxl
- Jinja2
- Pillow (optional for image resizing)
- pathlib
- shutil
- logging
- dataclasses

Frontend

- HTML5
- CSS3
- Vanilla JavaScript

Do not use:

- Node.js
- React
- Vue
- Angular
- Databases
- Web servers
- Build tools other than Python

Project Structure

A structure similar to the following is expected:

```
library/  
  
  src/  
    library_reader/  
    validators/  
    site_generator/  
    asset_manager/  
    models/  
  
  tests/  
  
  data/  
    books.xlsx  
  
  templates/  
    base.html  
    index.html
```

```
book.html

static/
  css/
  js/
  images/
  icons/

covers/

output/

generate.py

README.md
```

`generate.py` should be a lightweight entry point that orchestrates the build process.

Spreadsheet Format

The spreadsheet contains one row for each physical copy.

Suggested columns:

Column	Required
ID	Yes
Title	Yes
Author	Yes
ISBN	Optional
Genre	Yes
Series	Optional
Series Number	Optional
Publisher	Optional
Publication Year	Optional
Shelf	Optional
Status	Yes
Tags	Optional
Notes	Optional
Rating	Optional

Column	Required
Cover Image	Optional

Example ID:

BK000001

Requirements:

- IDs must be unique.
- Shelf is optional.
- Blank optional fields must not cause generation failures.
- If Shelf is blank, omit it from the generated page.

Validation

Before generation, validate:

- Duplicate IDs
- Missing required fields
- Invalid status values
- Missing cover images (warning only)
- Duplicate ISBNs (warning only)

Generation must stop if:

- Duplicate IDs exist
- Required fields are missing
- Invalid status values are found

Warnings should be displayed but should not prevent generation.

Display a validation summary before generation begins.

Build Behaviour

Running:

```
python generate.py
```

should:

1. Read the spreadsheet.
2. Validate the data.

3. Clean the output directory.
4. Copy static assets.
5. Generate JSON.
6. Generate individual book pages.
7. Generate the catalogue page.
8. Produce a build summary.

The build must be deterministic.

Running the generator multiple times without changing the spreadsheet should produce identical output (excluding filesystem timestamps where applicable).

Generated Output

The generator should create:

```
output/  
  
  index.html  
  
  books.json  
  
  books/  
  
    BK000001.html  
    BK000002.html  
    ...
```

The `output` directory should contain everything required for deployment.

Catalogue Page

The catalogue page should include:

- Responsive layout
- Cover thumbnails
- Book title
- Author
- Genre
- Shelf (when present)
- Availability status
- Search box
- Dark mode toggle
- Sorting
- Filtering
- Total number of books

- Number currently available

All searching and filtering should occur entirely within the browser using JavaScript.

Individual Book Pages

Each book page should contain:

- Cover image
- Title
- Author
- ISBN
- Genre
- Publisher
- Publication year
- Shelf (if supplied)
- Status
- Tags
- Rating
- Notes

Include navigation back to the catalogue.

Status Values

Support the following values:

- Available
- On Loan
- Reserved

Display each using visually distinct status badges.

Search

Search should match:

- Title
- Author
- ISBN
- Genre
- Publisher
- Tags

Results should update instantly while typing.

Sorting

Support sorting by:

- Title
 - Author
 - Publication Year
 - Recently Added (spreadsheet order)
-

Filtering

Support filtering by:

- Genre
- Status

Filters should work together.

Dark Mode

Implement dark mode using CSS variables.

Remember the selected theme using `localStorage`.

Responsive Design

Support:

- Desktop
- Tablet
- Mobile

Use modern CSS techniques such as Grid and Flexbox.

Do not use CSS frameworks.

Images

Cover images should be stored separately.

Example:

`covers/`

`BK000001.jpg`

`BK000002.jpg`

If a cover image is missing:

- Display a placeholder image.
- Issue a warning during generation.

Generation should continue.

Templates

Use Jinja2 templates.

Do not construct HTML using Python string concatenation.

Provide at minimum:

- `base.html`
 - `index.html`
 - `book.html`
-

JavaScript

Use vanilla JavaScript.

Separate functionality into logical modules, for example:

- `search.js`
- `filters.js`
- `sorting.js`
- `theme.js`

Avoid large monolithic scripts.

Testing

Provide a comprehensive automated test suite using pytest.

Tests should include:

Spreadsheet Reader

- Reads valid spreadsheets
- Handles optional fields correctly
- Reports missing required fields

Validation

- Duplicate IDs
- Duplicate ISBN warnings
- Invalid status values
- Missing cover image warnings

Generator

- Generates index.html
- Generates one page per book
- Generates books.json
- Copies assets correctly

Templates

Verify generated HTML contains expected:

- Titles
- Authors
- Navigation
- Metadata

Do not test visual styling.

Integration Test

Provide an end-to-end test that:

- Uses a sample spreadsheet
- Runs the generator
- Verifies all expected output files exist
- Confirms one HTML page is generated per book
- Completes without errors

Aim for approximately 80–90% coverage of the Python codebase.

Code Quality

Follow PEP 8.

Use:

- Type hints
- Dataclasses where appropriate
- pathlib instead of os.path
- Logging instead of print
- Modular architecture
- Docstrings

The project should also include configuration for:

- pytest
- ruff
- black
- mypy

Components should be independently testable.

Future Expansion

The architecture should make future features straightforward to add without major refactoring.

Planned future enhancements include:

- Deployment to GitHub Pages
- Borrow request workflow
- QR codes
- Author pages
- Series pages
- Reading statistics
- Multiple collections
- ISBN metadata lookup
- Import and export tools

These features are **not** part of Version 1.

Deliverables

The completed project should include:

- Complete Python source code
- Jinja2 templates
- CSS
- JavaScript
- Example spreadsheet
- Sample cover images
- Placeholder cover image

- Automated test suite
- README with installation and usage instructions

A new user should be able to clone the project, install the dependencies, place their spreadsheet into the `data` directory, and generate the entire website by running:

```
python generate.py
```

The resulting `output` directory should be immediately viewable locally by opening `index.html` and later deployable to GitHub Pages without modification.